

UNIVERSIDADE DE SÃO PAULO — USP ICMC

Trabalho Prático 1: Polinômios Esparsos

Curso: Bacharelado em Ciência de Dados
Disciplina: Estruturas de Dados

Integrantes do Grupo

- Vinicius Gesser Hsiao — N° USP: 17933148
- Cássio Kanomata Bones — N° USP: 16984384
- Adryann Henrique Oliveira Olivatti — N° USP: 17363240

1. Projeto da Solução

Estrutura Escolhida para Representação: Lista Duplamente Encadeada Ordenada de modo decrescente

1.1 Implementação Detalhada das Operações

Operação	Descrição	Lógica de Implementação
DEF nome k	Define nome (do polinômio) a partir das k linhas seguintes, cada uma com coef grau.	Aloca a estrutura para nome e lê k pares (coeficiente, grau), inserindo-os de forma ordenada por grau.
SOMA A B R	$R = A + B$	Percorre simultaneamente os polinômios A e B somando coeficientes de mesmo grau e criando os nós em R.
PROD A B R	$R = A * B$	Para cada termo (c, g) de A, multiplica por cada termo (c, g) de B e insere o produto ($c_a * c_b$, $g_a + g_b$) em R.
ADD A c g	Soma a A o termo de coeficiente c e grau g.	Busca o nó de grau g em A: se existir, soma c ao coeficiente existente (removendo o nó se o resultado for 0); se não existir, insere mantendo a ordenação.
ESCALA A c	Multiplica todo coeficiente de A pelo inteiro c.	Percorre todos os nós de A multiplicando cada coeficiente por c. Se $c = 0$, libera todos os nós e deixa o polinômio vazio.
COEF A g	Coeficiente de grau g em A.	Percorre A até encontrar o grau g (retorna o coeficiente c) ou passar do grau desejado (retorna 0).
REMOVE A g	Apaga de A o termo de grau g.	Busca o nó com grau g na estrutura de A, ajusta os ponteiros do nó anterior/seguinte e desaloca a memória do nó.
REMOVEMENOR A	Apaga de A o termo de menor grau.	Remove diretamente o último nó (tail) da lista encadeada ordenada por grau.
GRAU A	Maior grau com coeficiente não nulo.	Acessa o primeiro nó (head) da lista ordenada ou o

Análise comparativa das complexidades assintóticas em Notação Big-O, onde N e M representam o número de termos não nulos dos polinômios envolvidos, e L o número de termos não nulos do polinômio resultado R."

2.1 Justificativa da Escolha Final

Por que uma estrutura dinâmica, e não uma estrutura estática

A alocação dinâmica é adequada para uma lista duplamente encadeada ordenada porque permite que a memória seja alocada sob demanda, conforme novos elementos são inseridos, e liberada quando eles são removidos. Dessa forma, não é necessário definir previamente a quantidade máxima de elementos, evitando tanto o desperdício de memória quanto a limitação imposta por um tamanho fixo.

Por que uma estrutura ordenada, e não uma estrutura ordenada por inserção livre

O conjunto de comandos do trabalho é dominado por operações que se beneficiam diretamente de uma ordem. SOMA e PROD dependem de comparar termos de mesmo grau entre dois polinômios, o que se resolve em uma única varredura linear quando ambos os operandos já estão ordenados por grau. GRAU exige apenas consultar o extremo de maior grau, e REMOVEMENOR o extremo de menor grau, mantendo a lista ordenada, ambos os extremos são acessíveis em tempo constante através de ponteiros. IMPRIME não precisa fazer nada além de percorrer a lista já na ordem pedida pelo enunciado, e IMPRIMEINV percorre a mesma lista no sentido oposto. Isso torna qualquer estrutura ordenada estruturalmente superior a uma estrutura não ordenada para os comandos deste trabalho.

Por que lista encadeada, e não array ordenado

O array ganha exatamente uma operação: COEF, que se resolve em $O(\log N)$ por busca binária graças ao acesso indexado, contra $O(N)$ em qualquer lista encadeada, que só pode varrer sequencialmente. Esse é o único ponto em que o array vence de forma assintótica. Em contrapartida, toda operação que insere ou remove um termo no meio da estrutura, DEF quando há graus fora de ordem, ADD, REMOVE, e a própria SOMA e PROD ao construir o polinômio resultado termo a termo, tem o custo de deslocar todos os elementos posteriores à posição de inserção, um custo que a notação $O(N)$ esconde mas que na prática envolve mover memória contígua repetidamente. Numa lista encadeada, o mesmo evento é resolvido apenas religando ponteiros, sem mover nenhum dado já existente. Como o enunciado deixa explícito na seção 6 que graus podem chegar em qualquer ordem e a inserção intermediária é frequente (DEF com termos fora de ordem, ADD sucessivos, construção de SOMA e PROD), o custo de deslocamento do array pesa sobre mais operações e mais vezes por execução. A escolha por lista encadeada prioriza, portanto, o conjunto de operações que o trabalho de fato exercita com mais frequência, aceitando conscientemente que COEF deixa de ser logarítmico.

Por que duplamente encadeada, e não simplesmente encadeada

Ambas alcançam as mesmas complexidades assintóticas nas operações do trabalho. A escolha da variante dupla não se justifica, portanto, pela complexidade assintótica, a justificativa correta é de custo constante e de robustez de implementação. A operação

REMOVE(A, g), que localiza um grau arbitrário no meio da lista, precisa desconectar um nó de sua vizinhança, numa lista simples isso exige rastrear manualmente, durante a própria busca, um ponteiro auxiliar para o nó anterior, o que aumenta a complexidade do código e o risco de erros de borda. Numa lista dupla, o nó já carrega a referência ao seu antecessor, e a remoção se reduz a religar dois pares de ponteiros de forma direta e simétrica, sem lógica auxiliar. De modo semelhante, IMPRIMEINV percorre a lista em ordem crescente de grau partindo da cauda: numa lista dupla isso é uma travessia direta pelos ponteiros anterior; numa lista simples, a mesma operação exigiria percorrer a lista inteira do início empilhando referências, consumindo memória auxiliar proporcional a N que a lista dupla dispensa. Essas vantagens são de constante, não de ordem de grandeza, mas reduzem a superfície de bugs do código e evitam alocação adicional em uma operação. O custo dessa escolha é o de uma estrutura com mais informação por nó, cada termo carrega um ponteiro extra e cada inserção ou remoção precisa atualizar dois ponteiros em vez de um, o que se traduz em um fator constante levemente maior nessas operações.

3. Medição de Desempenho

Para testar o desempenho do programa, usamos o gerador .c, que foi configurado para criar diferentes quantidades de termos e de operações ESCALA. Foram realizados nove testes, começando com poucos termos e aumentando gradualmente até chegar a 50.000 termos. A quantidade de operações ESCALA foi aumentada na mesma proporção da quantidade de termos. Todos os testes foram realizados uma vez, utilizando o mesmo programa e o mesmo perfil de geração (faixas de grau e coeficiente definidas no gerador.c); cada execução usa uma sequência aleatória diferente, mas isso não afeta o padrão de crescimento observado, pois o custo depende do número de termos, não dos valores específicos sorteados.

Termos (N)	ESCALA	Linhas de Entrada	Tempo
10	4	30	0,001 s
30	12	78	0,001 s
100	40	246	0,001 s
300	120	726	0,001 s
1.000	400	2.406	0,003 s
3.000	1.200	7.206	0,031 s
10.000	4.000	24.006	0,434 s
30.000	12.000	72.006	5,465 s

Termos (N)	ESCALA	Linhas de Entrada	Tempo
50.000	20.000	120.006	25,562 s

Análise dos resultados. Nos testes menores, de até aproximadamente 1.000 termos, a diferença de tempo é muito pequena. Isso acontece porque, nesses casos, o tempo necessário para iniciar o programa e ler os dados acaba sendo mais relevante do que o tempo gasto pela estrutura de dados.

A diferença começa a ficar mais clara a partir de 3.000 termos. Conforme a quantidade de termos aumenta, o tempo cresce muito rapidamente. Por exemplo, ao passar de 3.000 para 10.000 termos, a quantidade de termos aumenta pouco mais de três vezes, mas o tempo aumenta de 0,031 s para 0,434 s. De 10.000 para 30.000 termos, o número de termos novamente aumenta cerca de três vezes, enquanto o tempo passa de 0,434 s para 5,465 s. Isso mostra que o programa não aumenta seu tempo de execução de forma proporcional à quantidade de termos. O tempo cresce cada vez mais rápido, indicando que existe uma parte do programa que fica muito mais pesada quando o número de termos aumenta.

O principal motivo está na criação dos polinômios. Para inserir cada novo termo, o programa procura na lista o lugar correto para colocá-lo. Como os graus dos termos são gerados em ordem aleatória, essa procura pode precisar percorrer uma grande parte da lista a cada nova inserção.

4. Ponto Fraco da Solução

A estrutura escolhida possui um ponto fraco identificável em cargas de trabalho dominadas pelo acesso aleatório. As operações frequentes de busca de coeficientes (como múltiplos comandos COEF em polinômios extensos) penalizam a lista encadeada por não haver atalho para o meio da estrutura. Nesse cenário específico, um *array* ordenado teria desempenho superior graças à possibilidade de aplicar busca binária. Da mesma forma, cenários com poucas modificações estruturais e muitas leituras repetidas reduziram o impacto do custo de deslocamento do *array*, tornando-o mais competitivo.

Para pequenos volumes de dados, esse custo de busca sequencial na lista é imperceptível. No entanto, ao lidar com dezenas de milhares de termos, a necessidade de percorrer a lista a cada nova inserção gera um gargalo expressivo durante a construção do polinômio. Esse comportamento fica evidente no processamento das maiores cargas: a entrada com 50.000 termos levou aproximadamente 25,6 segundos.

5. Casos de Teste Criados

Foram elaborados 4 casos de teste agrupados por tema, cobrindo todas as operações do programa e as 12 situações especiais exigidas. Cada caso é definido por um arquivo de

entrada (sequência de comandos) e a saída esperada correspondente.

5.1 Teste 1 — DEF e estrutura básica

Funções exercitadas: DEF, IMPRIME, IMPRIMEINV, GRAU, COEF.

Entrada	Saída Esperada
DEF A 3	$2x^3 5x^1 -1x^0$
2 3	$-1x^0 5x^1 2x^3$
5 1	$8x^2 1x^0$
-1 0	0
IMPRIME A	-1
IMPRIMEINV A	0
DEF D 3	-1
3 2	5
5 2	0
1 0	0
IMPRIME D	3
DEF E 2	
1 5	
-1 5	
IMPRIME E	
GRAU E	
DEF C 0	
IMPRIME C	
GRAU C	
COEF A 1	
COEF A 99	
COEF C 7	
GRAU A	
FIM	

Situações especiais cobertas: DEF com graus repetidos que se somam (polinômio D); coeficiente que zera o termo e some, ocorrendo durante o próprio DEF (polinômio E); DEF nome 0 gerando polinômio nulo (polinômio C); GRAU de polinômio nulo retornando -1; COEF de grau ausente retornando 0, inclusive em polinômio nulo.

5.2 Teste 2 — SOMA e PROD

Funções exercitadas: SOMA, PROD, DEF, IMPRIME.

Entrada	Saída Esperada
DEF A 3	$5x^1 3x^0$
2 3	$-4x^6 -10x^4 10x^3 20x^1 -4x^0$
5 1	$-2x^3 4x^0$
-1 0	0
DEF B 2	
-2 3	
4 0	
SOMA A B R	
IMPRIME R	
PROD A B P	
IMPRIME P	
DEF Z 0	
SOMA Z B G	
IMPRIME G	
PROD Z B F	

IMPRIME F FIM	
------------------	--

Cobre soma com cancelamento de termo de mesmo grau (o termo de grau 3 se anula entre A e B), soma e produto com operando nulo (Z vazio), e produto normal entre dois polinômios com múltiplos termos.

5.3 Teste 3 — ADD, REMOVE e REMOVEMENOR

Funções exercidas: ADD, REMOVE, REMOVEMENOR, GRAU, IMPRIME.

Entrada	Saída Esperada
DEF A 3	$2x^3 + 8x^1 - 1x^0$
2 3	$2x^3 + 8x^1 - 1x^0$
5 1	$8x^1 - 1x^0$
-1 0	$-1x^0$
ADD A 3 1	$-1x^0$
IMPRIME A	0
ADD A 0 2	-1
IMPRIME A	0
ADD A -2 3	
IMPRIME A	
REMOVE A 1	
IMPRIME A	
REMOVE A 99	
IMPRIME A	
REMOVEMENOR A	
IMPRIME A	
GRAU A	
DEF B 0	
REMOVEMENOR B	
IMPRIME B	
FIM	

Situações especiais cobertas: coeficiente que zera o termo e some, via ADD; REMOVE de grau ausente sem efeito; ADD de grau já existente somando ao coeficiente; ADD com coeficiente 0 sem efeito; REMOVEMENOR de polinômio nulo sem efeito. Também testa REMOVEMENOR normal e REMOVE de grau existente.

5.4 Teste 4 — ESCALA, redefinição de nome e LIBERA

Funções exercidas: ESCALA, DEF (redefinição), LIBERA, GRAU, IMPRIME.

Entrada	Saída Esperada
---------	----------------

DEF A 2	0
5 1	-1
-1 0	0
ESCALA A 0	$10 \cdot x^0$
IMPRIME A	0
GRAU A	$7 \cdot x^2$
ESCALA A 0	2
IMPRIME A	
DEF B 2	
5 1	
-1 0	
DEF B 1	
10 0	
IMPRIME B	
GRAU B	
DEF C 0	
LIBERA C	
DEF C 1	
7 2	
IMPRIME C	
GRAU C	
DEF D 2	
3 1	
2 0	
LIBERA D	
LIBERA Z	
FIM	

Situações especiais cobertas: ESCALA A 0 tornando A nulo, inclusive reaplicando ESCALA 0 sobre o polinômio já vazio; redefinição de um nome já usado descartando o conteúdo anterior (polinômio B); LIBERA seguido de DEF no mesmo nome, testado especificamente com um polinômio vazio no momento do LIBERA (polinômio C). Também cobre LIBERA de um polinômio com termos (D) e LIBERA de um nome inexistente (Z), que não deve ter efeito algum.